

Reproducibility Instructions

NYC Airbnb Market Intelligence - Data Engineering Assignment

Step-by-step setup guide: dependencies, environment, data download, and execution order.

1. Overview

This document describes how to set up the environment and reproduce the analysis pipeline for the NYC Airbnb Market Intelligence project. The project analyzes New York City short-term rental listings using the Inside Airbnb public dataset, covering data cleaning, exploratory analysis, statistical hypothesis testing, and price-prediction modeling.

Following the steps below in order will regenerate all processed data files, figures, statistical results, and model outputs from the raw source data.

1.1 Project Structure

```
airbnb-analysis/
├── data/
│   ├── raw/          # Downloaded source files (Inside Airbnb)
│   └── processed/     # Generated by src/clean.py
├── notebooks/
│   ├── 01_dataset_familiarization.ipynb
│   ├── 03_Data_Engineering_Challenges.ipynb
│   └── 04_Exploratory_Data_Analysis.ipynb
├── src/
│   ├── clean.py      # Data cleaning & validation
│   ├── eda.py        # Generates EDA figures
│   ├── stats_tests.py # Hypothesis tests (H1-H5)
│   └── model.py       # Price prediction models + SHAP
├── sql/
│   └── star_schema.sql # DuckDB star schema + example queries
├── reports/
│   └── figures/       # Generated charts (PNG)
├── docs/
│   ├── decisions_log.md
│   ├── ai_usage_disclosure.md
│   └── report_outline.md
├── README.md
└── requirements.txt
```

2. Dependencies

All Python dependencies are listed in requirements.txt at the project root. Install them with a single command (see Section 3.2).

2.1 Required Packages

Package	Purpose in this project
pandas	Data loading, cleaning, and transformation
numpy	Numerical operations supporting pandas and modeling
matplotlib	Base plotting library for EDA figures
seaborn	Statistical visualization (boxplots, scatterplots, etc.)
scipy	Hypothesis tests (Mann-Whitney U, ANOVA) in stats_tests.py
scikit-learn	Train/test split, Linear Regression, Random Forest, evaluation metrics
xgboost	Gradient-boosted price prediction model
shap	Model explainability (SHAP summary plot) for the XGBoost model
duckdb	Runs the star schema and analytical SQL queries
folium	Geospatial / map-based visualization
geopandas	Geospatial data handling for neighborhood-level analysis
jupyter	Runs the notebooks (01, 03, 04)

Exact file contents of requirements.txt:

```
pandas
numpy
matplotlib
seaborn
scipy
scikit-learn
xgboost
shap
duckdb
folium
geopandas
jupyter
```

Note: requirements.txt does not pin specific versions. For strict reproducibility, consider freezing versions (pip freeze > requirements-lock.txt) once a working environment is confirmed.

2.2 System-Level Requirements

- Python 3.9 or later
- pip (or conda) for package installation
- ~500 MB free disk space for raw and processed data files
- A terminal/command-line environment, or Google Colab (the existing notebooks are linked to Colab)

3. Environment Setup

3.1 Create an Isolated Environment (recommended)

Using a virtual environment avoids dependency conflicts with other Python projects.

Using venv:

```
python3 -m venv venv
source venv/bin/activate    # On Windows: venv\Scripts\activate
```

Or using conda:

```
conda create -n airbnb-analysis python=3.10
conda activate airbnb-analysis
```

3.2 Install Dependencies

From the project root (where requirements.txt is located), run:

```
pip install -r requirements.txt
```

This installs all twelve packages listed in Section 2.1.

3.3 Alternative: Google Colab

The project notebooks (01_dataset_familiarization.ipynb, 03_Data_Engineering_Challenges.ipynb, 04_Exploratory_Data_Analysis.ipynb) are stored as Google Colab notebooks. To reproduce using Colab instead of a local environment:

1. Open each notebook via its Colab link in Google Drive.
2. At the top of the notebook, add and run a setup cell installing any packages not preinstalled in Colab, for example:

```
!pip install duckdb shap geopandas folium xgboost
```

3. Mount Google Drive or upload the data/ folder so file paths referenced in the notebooks (e.g., data/raw/listings.csv.gz) resolve correctly.

Note: pandas, numpy, matplotlib, seaborn, scipy, scikit-learn, and jupyter are preinstalled in Colab by default; the !pip install step above only needs to cover the remainder.

4. Data Download

4.1 Source

All data originates from Inside Airbnb (<https://insideairbnb.com/>), a public, non-commercial dataset of Airbnb listings.

Field	Value
City	New York City
Scrape date used	2026-06-23
Download page	https://insideairbnb.com/get-the-data/

4.2 Files to Download

From the Inside Airbnb "Get the Data" page, locate the New York City section for the scrape date above and download the following four files:

File	Approx. Size	Contents
listings.csv.gz	~15 MB	One row per listing: price, location, room type, host info, reviews, etc.
calendar.csv.gz	~26 MB	Daily price/availability per listing for the next 365 days
reviews.csv.gz	~119 MB	Review dates per listing (listing_id, date)
neighbourhoods.csv	~5 KB	Lookup table of NYC boroughs and neighbourhoods

4.3 Placement

Place all four downloaded files in the `data/raw/` directory, without renaming them:

```
data/raw/listings.csv.gz
data/raw/calendar.csv.gz
data/raw/reviews.csv.gz
data/raw/neighbourhoods.csv
```

Note: `src/clean.py` reads directly from the `.gz` files (`pandas.read_csv` handles gzip transparently), so there is no need to manually decompress `listings.csv.gz` or `calendar.csv.gz` before running the pipeline.

Create the processed-data output directory if it does not already exist (the cleaning script will also create it automatically):

```
mkdir -p data/processed
```

5. Execution Order

The pipeline must be run in the following order, since each stage reads files produced by the previous one. All commands below are run from the project root with the virtual environment activated.

Step 1 — Clean and Validate Raw Data

```
python src/clean.py
```

What it does:

- Loads data/raw/listings.csv.gz and data/raw/calendar.csv.gz
- Parses the price field (strips \$ and commas, casts to float)
- Drops rows with price ≤ 0 or $> \$10,000/\text{night}$, and rows outside the NYC latitude/longitude bounding box
- Standardizes categorical fields (e.g., room_type)
- Derives host_tenure_years and price_per_bedroom
- Fills missing reviews_per_month with 0
- Writes data quality reports and the cleaned CSVs

Outputs:

```
data/processed/dq_report_listings_raw.csv
data/processed/listings_clean.csv
data/processed/calendar_clean.csv
```

Step 2 — Generate EDA Figures

```
python src/eda.py
```

Reads the cleaned CSVs from data/processed/ and produces five numbered figures into reports/figures/, matching the figure numbers used in the final report:

```
reports/figures/fig1_price_by_room_type.png
reports/figures/fig2_listings_per_host.png
reports/figures/fig3_price_vs_distance.png
reports/figures/fig4_reviewscore_vs_price.png
reports/figures/fig5_monthly_price_trend.png
```

Step 3 — Run Statistical Hypothesis Tests

```
python src/stats_tests.py
```

Runs five hypothesis tests (H1–H5) against the cleaned listings and calendar data and prints a results table (test statistic, p-value, effect size) to the console:

H1	Entire home/apt vs. private room price
-----------	--

H2	Superhost vs. non-superhost review scores
H3	>10 reviews vs. ≤10 reviews, price comparison
H4	Price differences across neighbourhood groups (ANOVA)
H5	Weekend vs. weekday price (calendar data)

Console output is intended to be copied directly into the Statistical Findings section of the project report.

Step 4 — Train Models and Generate SHAP Explainability Plot

```
python src/model.py
```

What it does:

- Builds a feature frame from listings_clean.csv (numeric + categorical features)
- Splits data 80/20 into train/test sets (random_state=42 for reproducibility)
- Trains and evaluates three model families: Linear Regression, Random Forest, and XGBoost (MAE, RMSE, MAPE)
- Saves a model comparison table
- Generates a SHAP summary plot explaining the XGBoost model's top price drivers

Outputs:

```
data/processed/model_comparison.csv
reports/figures/fig6_shap_summary.png
```

Step 5 — (Optional) Build the Analytical Star Schema in DuckDB

sql/star_schema.sql defines a small star schema (dim_listing, dim_host, fact_listing_day) over the processed CSVs, plus example analytical queries. This step is independent of Steps 1–4 above but depends on the same data/processed/ outputs from Step 1.

Run it via the DuckDB CLI:

```
duckdb < sql/star_schema.sql
```

Or from a Python/DuckDB connection:

```
import duckdb
con = duckdb.connect()
con.execute(open('sql/star_schema.sql').read())
```

Step 6 — Notebooks (Reference / Narrative Analysis)

The Jupyter notebooks contain the narrative write-up and supporting exploratory work corresponding to each pipeline stage. They can be run after the corresponding script(s) above, or independently if self-contained:

Notebook	Corresponds to
01_dataset_familiarization.ipynb	Schema documentation, key relationships, dataset limitations (Section 2 of report)
03_Data_Engineering_Challenges.ipynb	Cleaning/engineering narrative aligned with src/clean.py
04_Exploratory_Data_Analysis.ipynb	EDA narrative aligned with src/eda.py

Launch with:

```
jupyter notebook
```

then open the desired .ipynb file and run all cells top to bottom, or use the existing Colab links if working in Google Colab.

5.1 Quick Reference: Full Pipeline

```
# From project root, with virtual environment activated
pip install -r requirements.txt
mkdir -p data/processed
# (place downloaded raw files into data/raw/ first)

python src/clean.py
python src/eda.py
python src/stats_tests.py
python src/model.py
duckdb < sql/star_schema.sql # optional
```

6. Verifying a Successful Run

After completing Section 5, confirm the following files exist:

- data/processed/listings_clean.csv and data/processed/calendar_clean.csv
- data/processed/dq_report_listings_raw.csv
- Five figures (fig1–fig5) in reports/figures/
- Console output for hypothesis tests H1–H5 with p-values and effect sizes
- data/processed/model_comparison.csv and reports/figures/fig6_shap_summary.png

If any step fails, confirm that the previous step completed successfully and that its output files are present in data/processed/ — each script depends on the previous script's output rather than the raw data directly (except clean.py).

7. Known Limitations Affecting Reproducibility

- The dataset is a single-day scrape snapshot (2026-06-23), not live or continuously updated booking data; re-downloading at a later date will produce different raw files and therefore different downstream results.
- Inside Airbnb does not publish a separate hosts table — host-level attributes are derived from repeated rows in listings.csv rather than a normalized source.
- Row-drop counts during cleaning (price and lat/long validation in clean.py) are data-dependent and will vary slightly between scrape dates.
- Cloud deployment, containerization (Docker), and orchestration were out of scope for this assignment and are not part of this reproducibility guide.